

Taller 0: Introducción al Lenguaje **Go**

(c) **Diego Fernando Marín Lozano** - ver 2025-09-04

Objetivo: Este taller preparará tu entorno de desarrollo y te enseñará la sintaxis básica de Go necesaria para seguir el curso. Al final, serás capaz de escribir, compilar y ejecutar un programa simple de Go.

1. Instalación en **Ubuntu** o **WSL**

Desde una terminal sigue estos pasos:

1. Actualiza la lista de paquetes, y los paquetes:

```
sudo apt update  
sudo apt upgrade
```

2. Instala el paquete oficial de Go desde los repositorios de Ubuntu. Esto instalará una versión estable y funcional.

```
sudo apt install golang-go
```

3. Verifica la instalación

```
go version
```

2. Configuración del Espacio de Trabajo (Workspace)

Go tiene una forma particular de organizar el código, gracias a sus **módulos**.

1. Crea un directorio para tu proyecto del curso y navega hasta él:

```
mkdir -p concurrencia  
cd concurrencia
```

2. Inicializa un nuevo **módulo** de Go. Esto gestiona las dependencias de tu proyecto.

```
go mod init taller0
```

Esto creará un archivo `go.mod`. Es solo un archivo de texto que lleva el registro de las dependencias de tu proyecto.

3. Abre Visual Studio Code en este directorio:

```
code .
```

3. Primer Programa

Crea un nuevo archivo llamado `main.go` .

```
// main.go
package main // Todo programa ejecutable debe empezar con el paquete "main"

// Importamos paquetes que necesitamos.
// "fmt" es para formatear e imprimir texto.
import "fmt"

// La función main es el punto de entrada de nuestro programa.
// ¡Es donde todo comienza!
func main() {
    // Usamos el paquete fmt para imprimir un mensaje en la consola.
    fmt.Println("¡Hola, Mundo Concurrente!")

    // Declaramos una variable para contar.
    // "!=" es el operador de declaración corta (declara e inicializa).
    mensaje := "Este es el taller 0"

    // Llamamos a una función que definimos más abajo.
    imprimirMensaje(mensaje)

    // Usamos un bucle for (¡Go solo tiene 'for' para todo!).
    fmt.Println("Contando hasta 3:")
    for i := 1; i ≤ 3; i++ {
        fmt.Printf("Número %d\n", i) // Printf permite formateo con %d para números.
    }
}

// Definimos una nueva función llamada imprimirMensaje.
// Toma un argumento 'texto' de tipo string.
func imprimirMensaje(texto string) {
    fmt.Println("Mensaje de la función:", texto)
}
```

4. Compilación y Ejecución

Go hace que esto sea extremadamente simple.

1. Ejecutar directamente (Compila y ejecuta en un solo paso)

```
go run main.go
```

Verás la salida inmediatamente en tu terminal.

2. Compilar y luego ejecutar el binario

```
# Esto crea un archivo ejecutable con el nombre de tu directorio actual (modulo0)
go build

# Lista los archivos. Verás el nuevo ejecutable 'modulo0'
ls

# Ejecuta el programa compilado
./taller0
```

La ventaja de `go build` es que puedes llevar el ejecutable `modulo0` a cualquier máquina con el mismo sistema operativo y arquitectura y ejecutarlo, sin necesidad de tener Go instalado.

5. Conceptos Básicos Clave que Debes Recordar

- `package main` & `func main()` : Son obligatorios para crear un programa ejecutable.
- **Variables:** Se declaran con `var nombre tipo` o `nombre := valor` (dentro de funciones).
- **Funciones:** Se declaran con `func nombreFuncion(argumento tipo) tipoRetorno { }`.
- **Estructuras de Control:**
 - `for` : Es el único bucle. Puede ser un bucle `for i:=0; i<10; i++ { }`, un bucle `while` con `for condition { }`, o un bucle infinito `for { }`.
 - `if` : Similar a otros lenguajes. La condición no lleva paréntesis.
 - `switch` : Muy potente y se usa frecuentemente en Go.
- **Visibilidad:** Lo que es exportado (visible desde otros paquetes) debe empezar con **mayúscula** (`fmt.Println`, `MyFunction`). Lo que es privado, con **minúscula** (`myPrivateVariable`).

6. Ejercicios

Añadir la Función del Doble y Usarla

Implementa la función `calcularDoble`.

```
// ... (el código anterior se mantiene igual)

func main() {
    // ... (el código anterior de main se mantiene igual)
```

```

// --- NUEVO CÓDIGO PARA EL PASO 2 ---
numero := 7
resultado := calcularDoble(numero)
fmt.Printf("El doble de %d es %d\n", numero, resultado)
}

// ... (la función imprimirMensaje se mantiene)

// --- NUEVA FUNCIÓN PARA EL PASO 2 ---
// calcularDoble recibe un entero y devuelve su doble.
func calcularDoble(num int) int {
    return num * 2
}

```

Ejecuta el programa con `go run main.go` y asegurate que funciona correctamente antes de pasar al siguiente.

Leer Entrada del Usuario

Haz el programa más interactivo. Pídele un número al usuario para calcular su doble, en lugar de usar uno fijo.

```

package main

// Necesitamos importar el paquete 'bufio' para leer la entrada.
import (
    "bufio"
    "fmt"
    "os"
    "strconv" // También necesitamos este para convertir texto a número.
    "strings" // Para limpiar la entrada.
)

func main() {
    fmt.Println(";Hola, Mundo Concurrente!")
    mensaje := "Este es el módulo 0"
    imprimirMensaje(mensaje)

    fmt.Println("Contando hasta 3:")
    for i := 1; i ≤ 3; i++ {

```

```

        fmt.Printf("Número %d\n", i)
    }

    // --- NUEVO CÓDIGO PARA EL PASO 3 ---
    // Creamos un "lector" para la entrada estándar (el teclado).
    reader := bufio.NewReader(os.Stdin)

    fmt.Print("Por favor, ingresa un número para calcular su doble: ")
    // Leemos lo que el usuario escriba hasta que presione Enter.
    entrada, _ := reader.ReadString('\n')
    // Limpiamos el carácter de nueva línea y espacios en blanco.
    entrada = strings.TrimSpace(entrada)
    // Convertimos el string a un número entero (int).
    numero, err := strconv.Atoi(entrada)

    // MANEJO DE ERRORES: Si la conversión falla (ej. usuario escribe "hola").
    if err != nil {
        fmt.Println("Error: Debes ingresar un número válido.", err)
        return // Salimos de la función main (y del programa) anticipadamente.
    }

    resultado := calcularDoble(numero)
    fmt.Printf("El doble de %d es %d\n", numero, resultado)
}

// ... (las funciones imprimirMensaje y calcularDoble se mantienen igual)

```

Ejecuta el programa con `go run main.go` y asegurate que funciona correctamente antes de pasar al siguiente.

Organizar el Código en un Paquete Personalizado

Separa la lógica de cálculo en su propio paquete para aprender sobre estructura de proyectos.

1. Crea un nuevo directorio y archivo:

```

# Dentro del directorio 'taller0'
mkdir matematicas
touch matematicas/calculos.go

```

2. Edita `matematicas/calculos.go` :

```
// calculos.go
package matematicas // Declara que este archivo pertenece al paquete 'matematicas'

// La función debe empezar con MAYÚSCULA para ser exportada (visible desde main).
func CalcularTriple(num int) int {
    return num * 3
}
```

3. Modifica tu `main.go` para usar el nuevo paquete:

```
package main

import (
    "bufio"
    "fmt"
    "os"
    "strconv"
    "strings"
    // Importamos nuestro propio paquete.
    // La ruta es el nombre del módulo + la ruta al directorio del paquete.
    "github.com/tuusuario/curso-concurrencia/matematicas"
)

func main() {
    // ... (todo el código anterior de main se mantiene hasta el cálculo del doble)

    resultado := calcularDoble(numero)
    fmt.Printf("El doble de %d es %d\n", numero, resultado)

    // --- NUEVO CÓDIGO PARA EL PASO 4 ---
    // Usamos la función del nuevo paquete.
    resultadoTriple := matematicas.CalcularTriple(numero)
    fmt.Printf("El triple de %d es %d\n", numero, resultadoTriple)
}

// ... (las funciones calcularDoble e imprimirMensaje se mantienen)
```

Ejecuta y prueba `go run main.go`. Go descargará automáticamente las dependencias... pero como `matematicas` es un paquete local, simplemente funcionará. Verifica que ahora también se calcule y muestre el triple del número ingresado.

Añadir una Prueba Unitaria

Aprende a escribir tests en Go, una parte fundamental del desarrollo profesional.

1. Crea un archivo de test junto a `calculos.go` :

```
touch matematicas/calculos_test.go
```

2. Edita `matematicas/calculos_test.go` :

```
package matematicas // El test pertenece al mismo paquete

// Importamos el paquete "testing", necesario para tests.
import "testing"

// Una función de test debe empezar con 'Test' y recibir *testing.T.
func TestCalcularTriple(t *testing.T) {
    // Definimos un caso de prueba: entrada 4, resultado esperado 12.
    resultado := CalcularTriple(4)
    esperado := 12

    // Comprobamos el resultado.
    if resultado != esperado {
        // Informamos del error usando el parámetro t.
        t.Errorf("Test fallido: Se esperaba %d, pero se obtuvo %d", esperado, resultado)
    }
}
```

3. Ejecuta la prueba:

```
# Desde el directorio 'modulo0'
go test ./matematicas/
```

Deberías ver un mensaje que diga `PASS` si la prueba pasa correctamente.